

RAPPORT TECHNIQUE

Preuve de concept (POC)

Assistant intelligent de recommandation d'événements culturels

Système RAG : évènements culturels et publics parisiens

Client : Puls-Events

TITOUAN JEANNOT

Type de livrable	Rapport technique de POC
Domaine	RAG recherche augmentée par récupération
Périmètre	Évènements culturels à Paris
Source de données	API OpenDataSoft (OpenAgenda)
Dépôt	Github.com/leskimou/projet_système_rag

1. Objectifs du projet

Contexte : Puls-Events souhaite un assistant conversationnel capable de renseigner le public, en langage naturel, sur les événements culturels et publics.

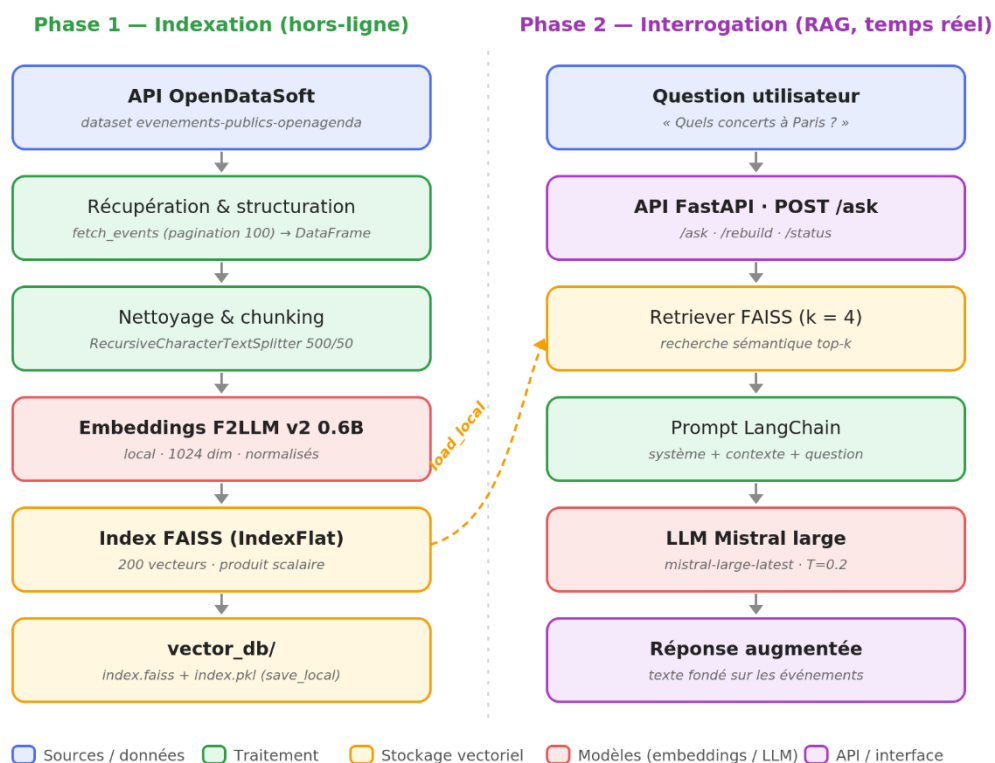
Problématique : un grand modèle de langage interrogé seul ignore l'agenda à venir et tend à inventer des détails ; l'approche RAG ancre chaque réponse sur des événements réels et actualisables sans réentraînement du modèle.

Objectif du POC : démontrer la faisabilité technique (récupérer, vectoriser et interroger des données ouvertes via un LLM), la valeur métier (des réponses utiles et sourcées) et la possibilité de mesurer objectivement la qualité.

Périmètre : ville de Paris, événements à partir du 1er janvier 2026, données ouvertes OpenDataSoft (200 événements dans l'index courant).

2. Architecture du système

Le système s'organise en deux phases : une phase d'indexation (hors-ligne) qui transforme les événements bruts en index vectoriel persistant, et une phase d'interrogation (temps réel) qui met en œuvre la chaîne RAG. Le schéma ci-dessous présente le flux global : données entrantes (API OpenDataSoft) → prétraitement et embeddings → base vectorielle FAISS → intégration LLM via LangChain → exposition par une API.



Composant	Technologie
Langage	<i>Python 3.11</i>
API / service web	<i>FastAPI + Uvicorn</i>
Orchestration RAG	<i>LangChain</i>
Embeddings	<i>sentence-transformers F2LLM-v2-0.6B (local)</i>
Base vectorielle	<i>FAISS (faiss-cpu)</i>
LLM de génération	<i>Mistral mistral-large-latest</i>
Évaluation qualité	<i>RAGAs</i>
Conteneurisation	<i>Docker</i>

3. Préparation et vectorisation des données

Source de données : API publique OpenDataSoft (dataset evenements-publics-openagenda), filtrée sur la ville de Paris et les événements dont la date de début est postérieure au 1er janvier 2026 ; récupération paginée par lots de 100 enregistrements.

Nettoyage : seuls les champs utiles sont conservés (titre, description, lieu, ville, dates, identifiant) ; les dates sont tronquées au jour, les champs manquants remplacés par une chaîne vide, et les plages « début = fin » réduites à une seule date.

Chunking : les descriptions sont découpées avec le RecursiveCharacterTextSplitter (taille 500, chevauchement 50) ; chaque fragment est préfixé d'un en-tête (titre, lieu, dates) pour rester autoporteur lors de la recherche.

Embedding : modèle F2LLM-v2-0.6B exécuté localement (sentence-transformers) ; vecteurs de dimension 2560, normalisés, avec encodage asymétrique (une instruction est ajoutée en tête des requêtes).

4. Choix du modèle NLP

Modèle sélectionné : Mistral mistral-large-latest pour la génération des réponses ; F2LLM-v2-0.6B pour la vectorisation.

Pourquoi ce modèle : bonne qualité rédactionnelle en français, intégration native dans LangChain (langchain-mistralai), inférence hébergée évitant d'héberger un LLM volumineux, et coût maîtrisé (température 0,2 et sortie limitée à 400 tokens).

Prompting : un prompt système impose de répondre uniquement à partir des événements récupérés, de détailler dates, lieux et informations pratiques, et de refuser poliment les questions hors-sujet (prompt complet en annexe B).

Limites du modèle : qualité plafonnée par le contexte récupéré, troncature possible des réponses à 400 tokens, quotas de l'API (erreurs 429), et ajout occasionnel d'informations générales non présentes dans la base.

5. Construction de la base vectorielle

FAISS : index plat IndexFlat (recherche exacte), avec un produit scalaire assimilé à la similarité cosinus grâce à la normalisation des vecteurs ; 200 vecteurs de dimension 1024, et k = 4 fragments retrouvés par requête.

Stratégie de persistance : sauvegarde sur disque via `save_local` (fichiers `index.faiss` et `index.pkl`, noms par défaut) et rechargement via `load_local` ; le dossier `vector_db/` n'est pas versionné car régénérable à la demande.

Métadonnées associées : pour chaque document sont conservés l'identifiant (uid), le titre, la description, la ville, le lieu et les dates de début et de fin ; elles sont restituées avec chaque fragment retrouvé.

6. API et endpoints exposés

Framework : FastAPI, servie par Uvicorn (validation automatique via Pydantic et documentation interactive).

Endpoint	Méthode	Rôle
<code>/ask</code>	POST	Pose une question et retourne la réponse augmentée
<code>/rebuild</code>	POST	Reconstruit l'index vectoriel en arrière-plan (paramétrable)
<code>/status</code>	GET	Renvoie l'état de la base et le nombre d'événements indexés

Format des requêtes / réponses : schémas Pydantic (`AskRequest` et `AskResponse`, `RebuildRequest`, `StatusResponse`) garantissant des échanges JSON validés.

Exemple d'appel : interrogation du chatbot avec curl :

```
curl -X POST http://127.0.0.1:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "Quels concerts ont lieu à Paris ?"}'
```

Tests effectués : suite pytest : tests unitaires (indexation et chaîne RAG, mockés) et tests fonctionnels de l'API via TestClient ; les tests d'évaluation RAGAs sont isolés car ils réalisent de vrais appels d'API.

Gestion des erreurs : /ask renvoie 503 si la base est absente ou en reconstruction, /rebuild renvoie 409 si une reconstruction est déjà en cours (verrou threading.Lock), et tout échec bascule l'état en error, consultable via /status.

7. Évaluation du système

Jeu de test annoté : 10 exemples (7 dans le périmètre, 3 hors-sujet : recette, météo, résultat sportif) ; chaque exemple comporte la question, la réponse de référence, la réponse produite et les contextes récupérés.

Métriques d'évaluation : framework RAGAs : fidélité (faithfulness), précision et rappel du contexte, et pertinence de la réponse ; le seuil d'acceptation est fixé à 0,70.

Résultats obtenus : le protocole est quantitatif : chaque test échoue si le score moyen de sa métrique passe sous 0,70. Qualitativement, le système identifie le bon événement dans le périmètre et refuse correctement les sujets non couverts ; point d'attention, il complète parfois avec des informations générales non présentes dans la base, ce que la métrique de fidélité permet de surveiller.

8. Recommandations et perspectives

Ce qui fonctionne bien : des réponses pertinentes et sourcées sur le périmètre, un garde-fou efficace contre le hors-sujet, une base restructurable à la demande et une industrialisation amorcée (Docker, tests, évaluation automatisée).

Limites du POC : volumétrie restreinte (200 événements, Paris uniquement), modèle d'embedding lourd (1 Go) et lent sur CPU, index exact non scalable à grande échelle, et dépendance aux quotas de l'API.

Améliorations possibles : étendre les données (autres villes, volume, nouveaux champs), ajouter un filtrage par métadonnées (date, catégorie), passer à un index approché (IVF ou HNSW), introduire un reranking et une mémoire conversationnelle, puis préparer un passage en production (configuration dédiée, gestion des secrets, observabilité, CI/CD et orchestration).

9. Organisation du dépôt GitHub

Le code est hébergé sur github.com/leskimou/projet_systeme_rag. Le dépôt sépare clairement le point d'entrée (API), la logique métier (utils), les tests et la configuration de déploiement.

```

projet_système_rag/
├── api.py                # API FastAPI (/ask, /rebuild, /status)
├── utils/
│   ├── indexer.py       # Récupération OpenDataSoft + index FAISS
│   ├── chatbot.py       # Chaîne RAG : retriever → prompt → Mistral
│   ├── embeddings.py    # Modèle d'embedding F2LLM-v2-4B (local)
│   └── ragas_ask.py     # Génération réponses/contextes (éval)
├── tests/
│   ├── unit/            # Tests unitaires (indexation, chaîne RAG)
│   └── functional/      # Tests d'API + évaluation RAGAs + jeu de test
├── vector_db/           # Index FAISS persistant (généré au runtime)
├── Dockerfile / entrypoint.sh # Image et démarrage de l'API
├── Makefile             # Commandes (build, run, test, rebuild...)
├── pyproject.toml / uv.lock # Dépendances et verrouillage (uv)
└── README.md            # Documentation d'installation et d'usage

```

api.py : point d'entrée FastAPI (schémas, endpoints, gestion d'état et reconstruction).

utils/ : logique métier, indexation, chaîne RAG, embeddings et génération pour l'évaluation.

tests/ : tests unitaires et fonctionnels (mockés) et évaluation RAGAs avec le jeu de test annoté.

Fichiers d'infrastructure : Dockerfile, entrypoint.sh, Makefile et configuration des dépendances (uv).

10. Annexes

Annexe A : Extraits du jeu de test annoté

Question	Réponse de référence
Y a-t-il une exposition sur la Corée au Musée Guimet ?	Oui, une visite libre de l'exposition « Silla : l'Or et le Sacré » le 23 mai 2026.
Quelle pièce est jouée au Théâtre des Bouffes du Nord en mars 2026 ?	« Lettres non-écrites » (Cie Lieux-dits / David Geselson), du 19 au 21 mars 2026.
Quelle est la recette de la tarte Tatin ?	Question hors périmètre : l'assistant indique ne pas traiter ce sujet.

Annexe B : Prompt système utilisé

```
SYSTEM_PROMPT = """Tu es un assistant spécialisé dans les événements culturels et publics sur Paris.
```

```

Tu dois conseiller les utilisateurs sur les événements à venir en te basant uniquement sur les données que tu retrouves dans ta base de connaissances.

```

Donne des réponses et descriptions détaillées des événements, incluant les dates, lieux, horaires et informations pratiques renseigné.

- Ne mentionne AUCUNE information absente des événements fournis.
- N'invente pas d'horaires, tarifs, adresses ou accès qui ne figurent pas dans la description.
- N'extrapolés pas le contenu supposé d'une exposition ou d'un événement.
- N'ajoute pas de liens ou URLs.
- Si une information n'est pas dans le contexte, dis simplement qu'elle n'est pas disponible.

Si tu ne trouves pas d'événements pertinents, réponds honnêtement que tu n'as pas d'information à ce sujet.

Si on te pose une question qui n'est pas liée aux événements culturels et publics sur Paris, réponds honnêtement que tu ne peux pas répondre à cette question et invite l'utilisateur à poser une question sur les événements culturels et publics sur Paris.

Événements pertinents :

```
{context}
""
```

Annexe C : Exemple de réponse (extrait JSON)

```
{
  "question": "Quelle pièce est jouée au Théâtre des Bouffes du Nord en mars 2026 ?",
  "answer": "En mars 2026, le Théâtre des Bouffes du Nord présente la pièce Lettres non-écrites – Compagnie Lieux-dits / David Geselson, du 19 au 21 mars 2026 (37 Bis Bd de la Chapelle, 75010 Paris).",
  "contexts": [
    "Titre: Lettres non-écrites | Lieu: Théâtre des Bouffes du Nord, Paris | Dates: 2026-03-19 → 2026-03-21"
  ]
}
```